



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

«Информатика и системы управления»

КАФЕДРА

«Теоретическая информатика и компьютерные технологии»

**Отчет о лабораторной работе № 3
по курсу «Операционные системы»
Список процессов в системе**

Студент: Федукон А. А.

Группа: ИУ9-42Б

Преподаватель: Брагин А.В.

Москва, 2025 г.

Содержание

Постановка задачи	2
Часть 1. Вывод списка процессов из драйвера в ReactOS	2
Часть 2. Вывод списка процессов из драйвера в NetBSD	3
Практическая реализация	3
Часть 1. Вывод списка процессов из драйвера в ReactOS	3
Часть 2. Вывод списка процессов из драйвера в NetBSD	5
Результаты	7
Часть 1. Вывод списка процессов из драйвера в ReactOS	8
Часть 2. Вывод списка процессов из драйвера в NetBSD	9
Выводы	9
Список литературы	10

Постановка задачи

Цель работы — разработать загружаемые модули ядра (драйвера) для ОС Windows или ReactOS и NetBSD, которые выводят в отладочный журнал список всех процессов в системе.

Часть 1. Вывод списка процессов из драйвера в ReactOS

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора процесса (PID) и идентификатора родительского процесса. Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты Task Manager (taskmgr.exe). Информацию о процессах получить из ядра с использованием функции `ZwQuerySystemInformation(SystemProcessInformation, ...)`. Выделение и осво-

бождение памяти для информации о процессах реализовать, используя функции `ExAllocatePool(PagedPool, . . .)` и `ExFreePool()`. Предусмотреть ситуацию, в которой запрошенный размер буфера, необходимый для размещения информации о процессах, может измениться в большую сторону между вызовами функции `ZwQuerySystemInformation`, и в этом случае произвести новое выделение памяти с освобождением старого буфера.

Часть 2. Вывод списка процессов из драйвера в NetBSD

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — `Image Name`), идентификатора процесса (`PID`) и идентификатора родительского процесса. Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты `ps`. Информацию о процессах получить из ядра, используя глобальную переменную `allproc`. Для этого необходимо включить заголовочные файлы:

```
#include <sys/module.h>
#include <sys/kernel.h>
#include <sys/proc.h>
```

Самостоятельно следует изучить проблему защиты от «грязного чтения» списка процессов в момент его изменения операционной системой (например, при создании нового процесса или удалении завершенного).

Практическая реализация

Часть 1. Вывод списка процессов из драйвера в ReactOS

В этой лабораторной работе я продолжил развитие проекта драйвера, начатого в предыдущих заданиях. Цель — реализовать модуль ядра для ReactOS, который выводит в отладочный лог список всех процессов в системе с указанием их `PID`, `PPID` и имени исполняемого файла.

Подготовка среды

Я использовал уже настроенную среду сборки ReactOS, созданную в лабораторной работе №1. Виртуальную машину запускал через virt-manager с использованием KVM, отказавшись от работы через консольный QEMU. Также я собрал дополнительный виртуальный внешний диск с помощью утилиты genisoimage, куда положил скомпилированный драйвер и подключил диск к виртуальной машине через интерфейс virt-manager.

Изменения по сравнению с lab2

Я скопировал содержимое папки drivers/lab2 в новую папку drivers/lab3. Внутри я переименовал файлы:

- lab2.c → lab3.c
- lab2.rc → lab3.rc

Я также отредактировал CMakeLists.txt внутри папки lab3, указав новые имена исходников. Не забыл добавить строку `add_subdirectory(lab3)` в общий файл drivers/CMakeLists.txt, чтобы новая директория участвовала в сборке системы [3].

Сборка и установка

Сборку драйвера выполнил командой:

```
ninja lab3
```

Затем создал новый ISO-образ с помощью:

```
genisoimage -o external.iso -b bootcd/bootcd.iso ReactOS/out
```

и подключил его как виртуальный CD-диск к машине в virt-manager, чтобы перенести драйвер в папку `C:\ReactOS\system32\drivers\`.

Ключевые моменты реализации lab3.c

В коде lab3.c реализована логика динамического выделения буфера под список процессов, используя `ExAllocatePoolWithTag` и повторные вызовы

ZwQuerySystemInformation, если буфера недостаточно. В цикле происходит обход всех структур SYSTEM_PROCESS_INFORMATION с выводом PID, PPID и имени образа:

```
DPRINT1("%-8u %-8u %wZ\n",  
        HandleToUlong(processInfo->UniqueProcessId),  
        HandleToUlong(processInfo->InheritedFromUniqueProcessId),  
        &processInfo->ImageName);
```

Также я указал фамилию в логах:

```
DPRINT1("(Fedukov lab3) Process list:\n");
```

и добавил её в имя устройства:

```
#define DEVICE_NAME L"\\Device\\FedukovLab3"  
#define SYMBOLIC_LINK_NAME L"\\DosDevices\\FedukovLab3"
```

Загрузка и проверка драйвера

После загрузки ReactOS я выполнил в командной строке:

```
sc create lab3driver binPath=C:\\ReactOS\\system32\\drivers\\lab3.sys type= kernel  
sc start lab3driver
```

Часть 2. Вывод списка процессов из драйвера в NetBSD

Работа велась в исходном дереве NetBSD, полученном в лабораторной работе №1. Я создал копию папки lab2 под именем lab3 в директории /usr/src/sys/modules, и переименовал соответствующие файлы.

Основные изменения касались реализации функции, выводящей список процессов, в файле lab3.c. Код использует следующие заголовки[3]:

```
#include <sys/module.h>  
#include <sys/systm.h>  
#include <sys/param.h>  
#include <sys/proc.h>  
#include <sys/kernel.h>
```

```
#include <sys/sysctl.h>
```

Ключевые моменты реализации

Модуль реализует стандартную точку входа `process_lister_modcmd`:

```
static int
process_lister_modcmd(modcmd_t cmd, void *arg)
{
    switch (cmd) {
        case MODULE_CMD_INIT:
            printf("Process Lister module loaded\n");
            list_processes();
            return 0;

        case MODULE_CMD_FINI:
            printf("Process Lister module unloaded\n");
            return 0;

        default:
            return ENOTTY;
    }
}
```

Основная функция модуля — `list_processes()` — выполняет безопасный обход списка всех процессов через `LIST_FOREACH`, с захватом и освобождением `proc_lock`:

```
static void
list_processes(void)
{
    struct proc *p;
    struct proc *pp;

    printf("\n%-8s %-8s %-20s %s\n", "PID", "PPID", "STATUS", "COMMAND");
```

```

printf("-----\n");

mutex_enter(&proc_lock);
LIST_FOREACH(p, &allproc, p_list) {
    if (p->p_stat == LSIDL)
        continue;

    pp = p->p_pptr;
    printf("%-8d %-8d %-20s %s\n",
        p->p_pid,
        pp ? pp->p_pid : 0,
        p->p_stat == LSRUN ? "RUNNING" :
        p->p_stat == LSSTOP ? "STOPPED" :
        p->p_stat == LSSLEEP ? "SLEEPING" :
        p->p_stat == LSZOMB ? "ZOMBIE" : "UNKNOWN",
        p->p_comm);
}
mutex_exit(&proc_lock);
printf("-----\n\n");
}

```

Сборка и загрузка модуля

Сборка модуля выполнялась из директории `/usr/src/sys/modules/lab3`:

```

cd /usr/src/sys/modules/lab3
make

```

Полученный файл `lab3.kmod` загружался командой:

```

modload ./lab3.kmod

```

Результаты

При загрузке модуль выводит в лог список всех процессов. Сравнение результата со встроенными программами для мониторинга процессов показало корректное совпадение PID, PPID и имён процессов.

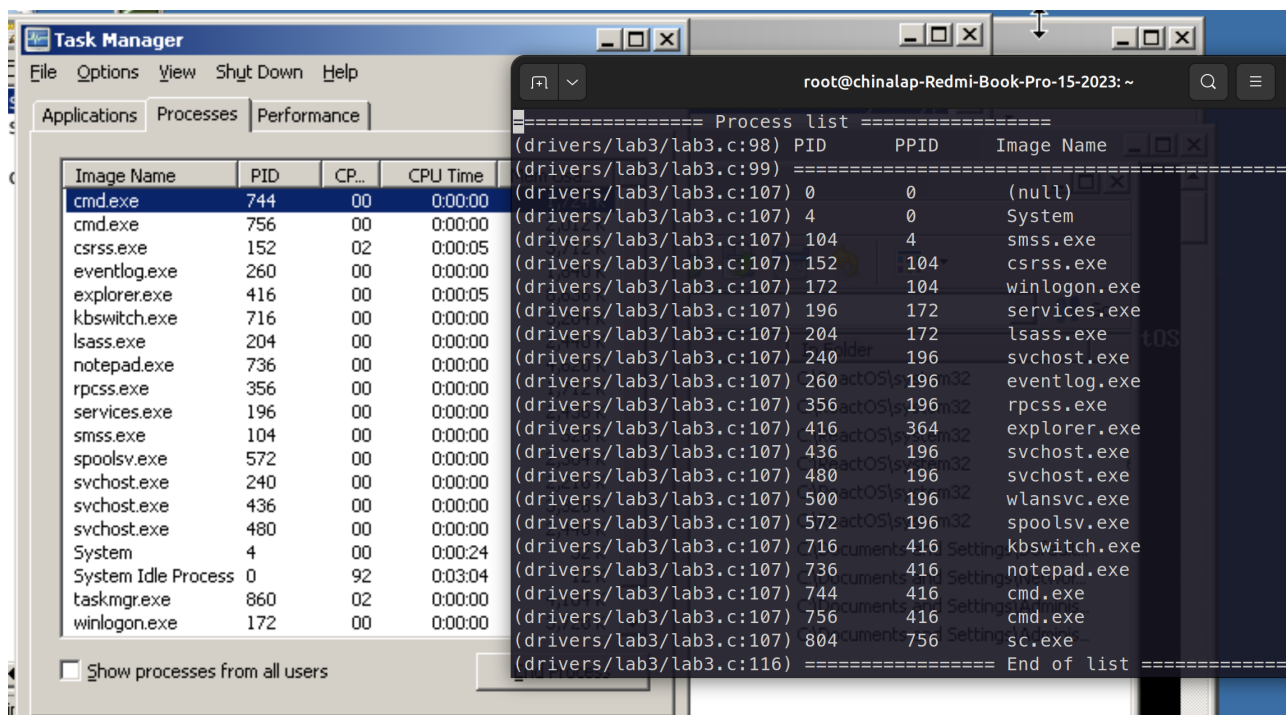


Рис. 1 — Процессы в reactos


```
ssh root@192.168.122.54
hostname# modload ./LAB2.KMOD
hostname# dmesg | tail -30
[ 525.394249] WARNING: module error: module '-h' not found
[ 582.213759] driver LAB2 FEDUK0Vdriver LAB2 FEDUK0Vdriver LAB2 FEDUK0VProcess
Lister module loaded (STRING RegistryPath)

[ 4562.629623] PID      PPID      STATUS      COMMAND
[ 4562.629623] -----
[ 4562.639648] 1066      858        RUNNING    modload
[ 4562.639648] 858        895        RUNNING    sh
[ 4562.639648] 895        561        RUNNING    sshd
[ 4562.639648] 823        813        RUNNING    sh
[ 4562.639648] 816        1          RUNNING    getty
[ 4562.639648] 994        1          RUNNING    getty
[ 4562.639648] 813        1          RUNNING    login
[ 4562.639648] 1832       1          RUNNING    getty
[ 4562.639648] 2020       1          RUNNING    cron
[ 4562.639648] 859        1          RUNNING    inetd
[ 4562.649772] 1276       837        RUNNING    qmgr
[ 4562.649772] 833        837        RUNNING    pickup
[ 4562.649772] 837        1          RUNNING    master
[ 4562.649772] 561        1          RUNNING    sshd
[ 4562.649772] 606        1          RUNNING    powerd
[ 4562.649772] 605        1          RUNNING    ntpd
[ 4562.649772] 303        1          RUNNING    modload
```

Рис. 2 — Процессы в netbsd

Выводы

В ходе выполнения лабораторной работы №3 я получил ценный практический опыт разработки загружаемых модулей ядра в двух различных операционных системах: ReactOS и NetBSD.

Работая с ReactOS, я научился использовать внутренние функции ядра Windows-совместимой архитектуры, такие как ZwQuerySystemInformation, а также работать с системными структурами и отладкой на уровне ядра. Мне было особенно интересно наблюдать, как работает взаимодействие с низкоуровневыми API, и как драйвер может получить доступ к списку процессов, не вызывая сбоев в системе.

При разработке драйвера для NetBSD я познакомился с совершенно другим подходом к модульности ядра, структурой процесса и системой синхронизации. Я понял важность захвата мьютексов (в частности, `proc_lock`) при доступе к

общим структурам ядра, таким как список процессов. Также я освоил макрос `LIST_FOREACH` и работу с элементами типа `struct proc`.

Одним из самых интересных аспектов было сравнение двух подходов к решению одной и той же задачи на разных архитектурах. Это дало мне понимание различий в дизайне ОС и укрепило мои навыки системного программирования.

Кроме того, я научился создавать ISO-образы с модулями, чтобы подключать их как виртуальные внешние диски в QEMU. Это оказалось удобным способом доставки и тестирования модулей внутри виртуальной машины.

В целом, лабораторная работа была насыщенной и полезной. Я стал лучше понимать внутреннее устройство операционных систем и принципы работы ядра.

Список литературы

- [1] Официальная документация ReactOS
(<https://reactos.org/wiki/>)
- [2] Официальная документация NetBSD
(<https://www.netbsd.org/docs/>)
- [3] Учебно-методическое пособие курса "Операционные системы"
(<https://press.bmstu.ru/catalog/item/8226/>)